

RELATÓRIO TÉCNICO IA: PLATAFORMA D&O

Fase 1: Sintetização e Ingestão de Dados

(Etapa 1)

Geramos dois arquivos de apólices espelhos em formato PDF nativo utilizando programação em Python. Injetamos clausulados reais registrados na SUSEP (padrões Chubb e Tokio Marine) com dados cadastrais fictícios para simular perfeitamente a entrada do sistema sem violar regras da LGPD ou segredos comerciais.



Fase 2: Extração e Organização Cognitiva Multimodal

Os arquivos binários dos PDFs foram enviados via API diretamente para o modelo fundacional de IA. O modelo realizou a leitura visual nativa e interpretou o texto livre e as tabelas regulatórias, limpando caracteres desnecessários e convertendo todo o conhecimento em um arquivo padronizado no formato JSON estruturado.



3: Banco Dados

Criamos o arquivo físico de banco de dados e desenvolvemos o script responsável por receber o JSON traduzido da IA. O código distribuiu cirurgicamente as informações em colunas estritas e tipadas (valores reais, textos e números inteiros), congelando o conhecimento de forma permanente.



Fase 4: Camada de Leitura e Motor Lógico de Auditoria

Construímos as queries SQL para recuperar as informações na memória local do computador e desenvolvemos os algoritmos matemáticos determinísticos em Python Puro para cruzar os dados, calculando lacunas de coberturas e disparidades de custos de defesa.



Fase 5: Conexão Frontend e Interface Conversacional RAG

Consolidamos toda a lógica na interface web em Modo Escuro. Integramos os cartões de visualização dinâmica e criamos o assistente virtual baseado em RAG local, permitindo que o usuário tire dúvidas sobre as cláusulas através de uma barra de chat interativa em tempo real.



ARQUITETURA DA SOLUÇÃO

O sistema foi desenhado sob o padrão de Arquitetura Desacoplada e Modular baseada em RAG Local (Retrieval-Augmented Generation). O ecossistema é dividido estritamente em três camadas independentes:

Camada de Ingestão e Inteligência (Nuvem): Responsável por quebrar a barreira do documento não estruturado e transformá-lo em dados rotulados.

Camada de Armazenamento e Lógica (Local): Responsável pela governança, persistência relacional estável e processamento de regras matemáticas imutáveis.

Camada de Visão e Interação (Frontend): Responsável por exibir a matriz de dados normalizada e intermediar a conversa do usuário com o contexto injetado.



3. TECNOLOGIAS UTILIZADAS

Alinhado com a premissa de viabilidade comercial e custo de desenvolvimento zero, a stack do MVP utiliza exclusivamente ferramentas gratuitas:

Linguagem Base: Python 3.11 (Isolada em Ambiente Virtual venv).

Engine Multimodal (IA): API SDK Oficial google-genai conectada ao modelo fundacional Gemini 3.6 Flash [PNCP].

Banco de Dados: SQLite3 (Nativo, relacional e em formato de arquivo local).

Engine Gráfica de Interface: Streamlit Framework (Open-source para apps de dados).

Modelagem de Documentos de Entrada: ReportLab PDF Library.

Manipulação Matricial de Dados: Pandas Library.



4. DESCRIÇÃO DOS AGENTES DESENVOLVIDOS

A autonomia do pipeline foi dividida entre três agentes lógicos principais com responsabilidades bem definidas:

Agente de Ingestão e Estruturação (Script: `extracao_gemini.py`): Atua como o leitor inicial. Sua função é receber o PDF bruto, acionar a visão computacional do modelo na nuvem e garantir o retorno estrito de um objeto JSON válido, tratando e limpando erros numéricos.

Agente de Persistência e Governança (Script: `banco_dados.py`): Atua como o guardião dos dados. Ele cria as tabelas SQL, decodifica o JSON e realiza a gravação do tipo *Upsert* (insere ou substitui se for duplicado), garantindo a integridade relacional.

Agente Consultivo de Interface (Script: app_streamlit.py): Intermedeia a experiência do usuário. Ele executa as consultas SQL, roda as regras matemáticas de assimetrias e gerencia o chat de inteligência artificial coletando a pergunta do usuário e injetando o contexto do banco de dados local nas chamadas da IA.



5. JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS

Substituição do OCR Tradicional por IA Multimodal Nativa: OCRs comuns locais (como Tesseract) costumam falhar grosseiramente ao ler tabelas com grades e linhas, gerando strings corrompidas. Enviar o arquivo binário direto para o Gemini 3.6 Flash aproveita a visão de máquina integrada da IA, que compreende a geometria do documento e entrega os dados com erro zero de transcrição [PNCP, Esclarecimentos - Telebras].

Eliminação de Frameworks Intermediários (LangChain): Modelos modernos possuem suporte nativo para Saídas Estruturadas (*Strict JSON Output*). Usar o SDK puro da Google reduz o tamanho do software, diminui a latência de resposta e remove códigos desnecessários (*Overengineering*).

Armazenamento em SQLite em vez de Chamadas de IA em Tempo Real: Chamar a IA toda vez que o usuário atualiza a página geraria custos absurdos de tokens e lentidão de 4 segundos por clique. Salvar no banco relacional local reduz o tempo de resposta da interface do Streamlit para menos de **0,001 segundos**, economiza a cota da API e protege o cliente corporativo contra quedas de internet.

Uso de Lógica de Recuo Exponencial (Retry Logic): Na camada gratuita, erros HTTP 503 (servidor indisponível) são normais em horários de pico [PNCP, Esclarecimentos - Telebras]. Criar o laço de repetição com pausas progressivas no código Python blinda a interface e garante o funcionamento estável do sistema do cliente sem quebrar a tela.



6. LIMITAÇÕES CONHECIDAS

Dependência da Cota da Camada Gratuita (Free Tier): O sistema fica sujeito aos limites diários de requisições impostos pelo Google AI Studio (15 acessos por minuto). Picos extremos de uso global na nuvem podem forçar o acionamento do recuo de tempo do nosso código, gerando pequenas pausas de segundos na resposta do chat.

Uso de Dados na Camada Aberta: Por utilizar chaves de testes gratuitas, os dados enviados em tempo real no chat passam pelos servidores externos da Google, exigindo o cuidado de mascarar dados pessoais críticos dos diretores (como CPF ou senhas) antes do envio.

